

UD6. Capa de transporte

Módulo: Planificación y administración de redes



Unión Europea

Fondo Social Europeo
El FSE invierte en tu futuro

Germán Gascón Grau
g.gascongrau@edu.gva.es



La capa de transporte es la encargada de la **comunicación extremo a extremo entre aplicaciones**, proporcionando servicios como fiabilidad, control de flujo y multiplexación, que la capa de red no ofrece.

La capa de transporte añade el **puerto** para **distinguir** entre los **diferentes destinos dentro de un mismo host**.

La misión fundamental de la capa 4 es especificar qué **aplicación** del destino recogerá el mensaje. Cada aplicación utiliza un puerto distinto para la recepción y envío de mensajes.

Introducción



SÍMIL: IP = DIRECCIÓN DEL EDIFICIO
PUERTO = NÚMERO DEL PORTAL

**1. CON LA IP LLEGAMOS
A LA CALLE**

IP: 192.168.1.10

LLEGAMOS AL EDIFICIO
EN ESA DIRECCIÓN



**2. CON EL PUERTO (NÚMERO DEL PORTAL)
SELECCIONAMOS EL SERVICIO**

PUERTO: 80

ENTRAMOS POR LA PUERTA 80
Y ACCEDEMOS AL SERVICIO
CORRESPONDIENTE



IP (DIRECCIÓN)
Nos lleva al edificio
(la calle)



EDIFICIO
Contiene muchos servicios
(médicos, abogados, etc.)



PUERTO (NÚMERO DEL PORTAL)
Selecciona el servicio específico
(número de puerta o interfono)





Capa de red (IP)

- Se encarga de llevar paquetes de un host a otro
- No garantiza orden, entrega ni fiabilidad

Capa de transporte

- Añade control y lógica sobre esa comunicación
- Hace que los datos lleguen correctamente a la aplicación adecuada

IP conecta máquinas, la capa de transporte conecta aplicaciones.



Unidad de datos: el segmento

La capa de transporte trabaja con segmentos (en TCP) o datagramas (en UDP)

Estos se encapsulan dentro de paquetes IP.

1. APLICACIÓN



Datos de aplicación
(Ej: solicitud HTTP)

Los datos se generan en la aplicación

2. TRANSPORTE (Capa 4)



CABECERA TCP
(Puertos, control, secuencia, etc.)

Datos de aplicación
(Ej: solicitud HTTP)

Se añade la cabecera de transporte para control extremo a extremo

Segmento (TCP)

3. RED (Capa 3)



CABECERA IP
(Direcciones IP, TTL, protocolo, etc.)

CABECERA TCP
(Puertos, control, secuencia, etc.)

Datos de aplicación
(Ej: solicitud HTTP)

Se añade la cabecera IP para el enrutamiento entre redes

Paquete IP (Datagrama)

4. ENLACE (Capa 2)



CABECERA DE ENLACE
(Direcciones MAC, control de errores, etc.)

CABECERA IP
(Direcciones IP, TTL, protocolo, etc.)

CABECERA TCP
(Puertos, control, secuencia, etc.)

Datos de aplicación
(Ej: solicitud HTTP)

TRÁILER DE ENLACE
(FCS / Control de errores)

Se añade la cabecera y el tráiler de enlace para la entrega en el medio físico

Trama (Frame)

10101011101010000101010101000010101

La trama se transmite como bits por el medio físico

MEDIO FÍSICO

Transporte

- Detectar errores
- Reordenar paquetes
- Evitar pérdidas de datos
- Adaptar la velocidad de envío
- Gestionar múltiples aplicaciones simultáneamente

Multiplexación y demultiplexación



Permite que múltiples aplicaciones usen la red al mismo tiempo.

Multiplexación (emisor):

- Junta datos de distintas aplicaciones
- Los etiqueta con un puerto

Demultiplexación (receptor):

- Recibe segmentos
- Los entrega a la aplicación correcta según el puerto

Ejemplo:

- Navegador Web → puerto 443
- Spotify → puerto 4070

Ambos usan la red simultáneamente



s en la

Mecanismos:

- Checksum
- ACK
- Retransmisión



Evitar que el emisor envíe datos más rápido de lo que el receptor puede procesar.

Cómo funciona

- El receptor indica cuánto puede recibir
- El emisor adapta su velocidad

Habitualmente se utiliza el mecanismo de **Ventana deslizante** (sliding window)



Concepto de puerto



Un puerto es un número (0–65535) que identifica una aplicación o servicio dentro de un sistema.

Rango	Tipo	Uso
0 – 1023	Bien conocidos	Servicios estándar
1024 – 49151	Registrados	Aplicaciones
49152 – 65535	Efímeros (dinámicos)	Clientes



Tipos de puertos



Puertos bien conocidos [0 - 1023]

- Son puertos reservados para servicios estándar de Internet.
- Permiten que cualquier cliente sepa automáticamente a qué puerto conectarse.
- Ejemplos: HTTP -> 80, HTTPS -> 443, DNS -> 53, SSH -> 22

Puertos registrados [1024 – 49151]

- Puertos asignados a **aplicaciones concretas**, pero no obligatorios.
- Ejemplos: PostgreSQL -> 5432, Minecraft -> 25565

Puertos dinámicos [49152 = 65535]

- Puertos temporales que el sistema asigna automáticamente.
- Cuando una aplicación inicia una conexión el sistema le asigna un puerto libre.
- Son necesarios para que un mismo cliente pueda abrir muchas conexiones simultáneas, por ejemplo varias pestañas del navegador.

En el siguiente enlace puedes encontrar la lista completa de puertos con su servicio asociado:

<http://www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.xhtml>





En la capa de transporte tenemos principalmente 2 protocolos:

- **TCP**
 - Fiable
 - Orientado a conexión
- **UDP**
 - Rápido
 - Sin conexión ni garantías

TCP (Transmission Control Protocol)



Características principales

- **Orientado a conexión.**

Es necesario establecer una conexión previa entre las dos máquinas antes de poder transmitir ningún dato

- **Fiable**

Garantiza la entrega en destino de la información enviada

- **Entrega ordenada**

Aunque los segmentos pueden fragmentarse y seguir diferentes caminos, TCP garantiza que serán entregados al destinatario en el orden correcto.

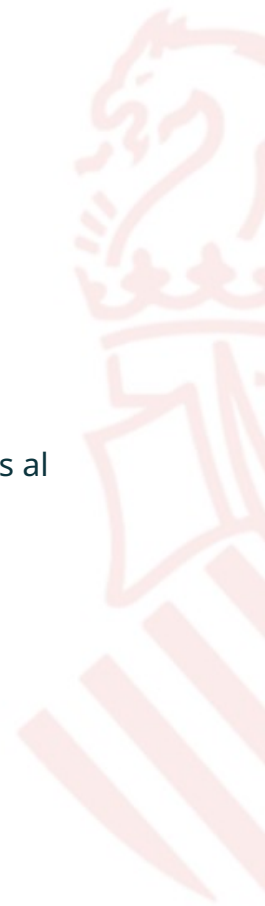
- **Control de flujo y congestión**

Adapta el flujo de emisión/recepción en función de la capacidad de emisión/recepción.

- **Más lento**

Debido al establecimiento de conexión y a todos los controles, es un protocolo con una latencia elevada.

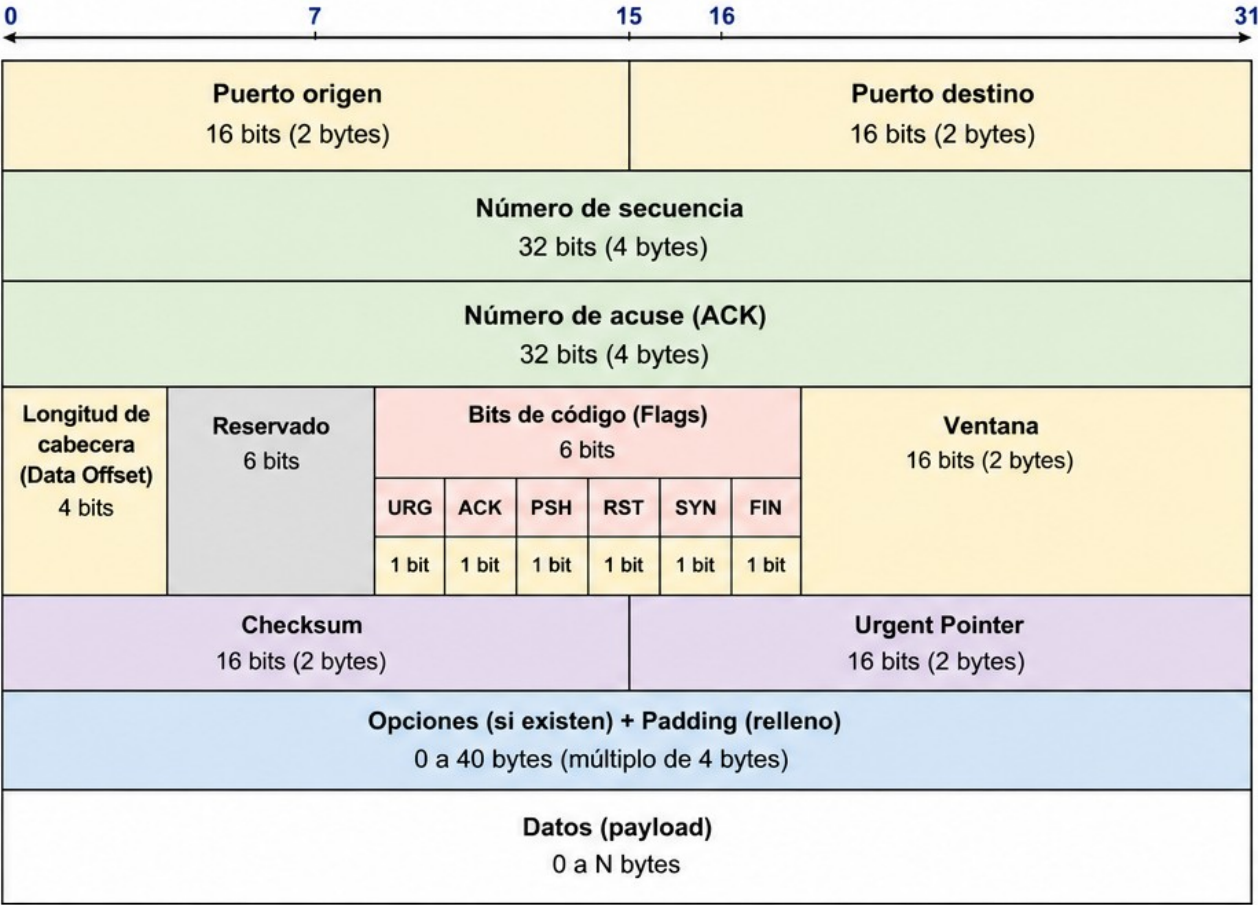
Ejemplos de uso: Web, Email, FTP



Formato de un segmento TCP



Formato de un segmento TCP



Tamaño

2 bytes

4 bytes

4 bytes

2 bytes

4 bytes

0 a 40 bytes

0 a N bytes

Descripción de los campos



Puerto origen / destino:
Identifican los procesos de origen y destino.



Número de secuencia:
Número del primer byte de datos en este segmento.



Número de acuse (ACK):
Número del siguiente byte que se espera recibir.



Longitud de cabecera (Data Offset):
Indica el tamaño de la cabecera en palabras de 32 bits. Valor mínimo 5 (20 bytes), máximo 15 (60 bytes).



Reservado:
Debe estar en 0 (para uso futuro).



Bits de código (Flags):
Controlan la conexión y el flujo.



Ventana:
Control de flujo. Indica cuántos bytes puede recibir el emisor.



Checksum:
Detecta errores en la cabecera y los datos.



Urgent Pointer:
Indica el final de los datos urgentes (si el flag URG está activo).



Opciones:
Parámetros adicionales (opcional).



Datos (payload):
Información de la capa de aplicación.

Formato de un segmento TCP



- **Puerto fuente** (16 bits). Puerto de la máquina origen. Al igual que el puerto destino es necesario para identificar la conexión actual.
- **Puerto destino** (16 bits). Puerto de la máquina destino.
- **Número de secuencia** (32 bits). Indica el número de secuencia del primer byte que transporta el segmento.
- **Número de acuse de recibo** (32 bits). Indica el número de secuencia del siguiente byte que se espera recibir. Con este campo se indica al otro extremo de la conexión que los bytes anteriores se han recibido correctamente.
- **HLEN** (4 bits). Longitud de la cabecera medida en múltiplos de 32 bits (4 bytes). El valor mínimo de este campo es 5, que corresponde a un segmento sin datos (20 bytes).
- **Reservado** (6 bits). Bits reservados para un posible uso futuro.
- **Bits de código** o indicadores/flags (6 bits). Los bits de código determinan el propósito y contenido del segmento. Cada uno de los bits representa: URG, ACK, PSH, RST, SYN y FIN respectivamente.



Significado de los bits de código o flags



Flag	Nombre	Función	Uso
URG	Urgent	Datos urgentes	En Aplicaciones interactivas. Poco uso. Indica qué parte de los datos es urgente.
ACK	Acknowledgment	Confirmación	Confirmar recepción de datos. El campo ACK contiene el siguiente byte esperado.
PSH	Push	Entrega inmediata	Indica que los datos deben ser entregados a la aplicación sin esperar a llenar los buffers. Usado frecuentemente.
RST	Reset	Reset conexión	Cierre abrupto de la conexión. Útil para indicar: errores graves, puerto cerrado y conexión inválida.
SYN	Synchronize	Inicio conexión	Para iniciar una conexión sincronizando números de secuencia.
FIN	Finish	Cierre conexión	Cierre ordenado cuando un extremo quiere cerrar la conexión.

Establecimiento de una conexión



Antes de transmitir cualquier información utilizando el protocolo TCP es necesario **abrir una conexión**.

Un extremo hace una apertura pasiva y el otro, una apertura activa. El mecanismo utilizado para establecer una conexión consta de tres vías (Three-Way Handshake)

- Cliente → SYN
- Servidor → SYN+ACK
- Cliente → ACK

Con esto ambos extremos **sincronizan números de secuencia** y quedan listos para enviar datos.



Establecimiento de una conexión



1. Cliente → SYN (El cliente quiere iniciar la conexión)

Cliente → Servidor: SYN

Seq = x

Significa: quiero conectar y empiezo con número de secuencia x.

2. Servidor → SYN + ACK (El servidor responde)

Servidor → Cliente: SYN + ACK

ACK = x + 1

Seq = y

Significa: acepto la conexión, confirmo tu SYN y mi número inicial es y

3. Cliente → ACK (El cliente finaliza)

Cliente → Servidor: ACK

ACK = y + 1

Significa: confirmo tu SYN

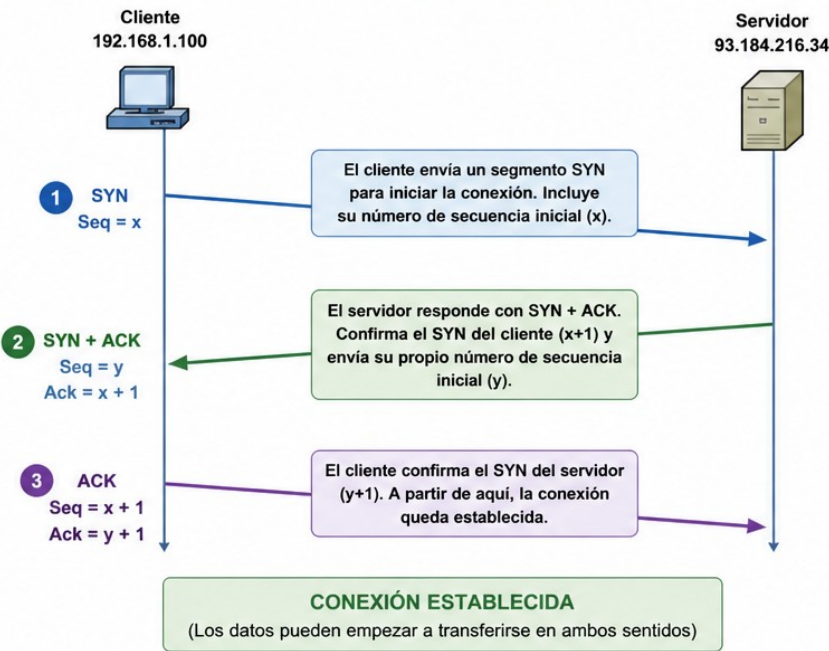


Establecimiento de una conexión



Establecimiento de una conexión TCP: Three-Way Handshake

Ejemplo: Cliente (192.168.1.100:51514) se conecta a Servidor (93.184.216.34:80)



Detalles de los segmentos (capturados con Wireshark)

1 Paquete 1: Cliente → Servidor (SYN)

192.168.1.100:51514 → 93.184.216.34:80

TCP

Flags: [SYN]

Sequence Number: 0 (x)

Acknowledgment Number: 0

Longitud de cabecera: 20 bytes (sin opciones)

URG	ACK	PSH	RST	SYN	FIN
0	0	0	0	1	0

2 Paquete 2: Servidor → Cliente (SYN + ACK)

93.184.216.34:80 → 192.168.1.100:51514

TCP

Flags: [SYN, ACK]

Sequence Number: 0 (y)

Acknowledgment Number: 1 (x + 1)

Longitud de cabecera: 20 bytes (sin opciones)

URG	ACK	PSH	RST	SYN	FIN
0	1	0	0	1	0

3 Paquete 3: Cliente → Servidor (ACK)

192.168.1.100:51514 → 93.184.216.34:80

TCP

Flags: [ACK]

Sequence Number: 1 (x + 1)

Acknowledgment Number: 1 (y + 1)

Longitud de cabecera: 20 bytes (sin opciones)

URG	ACK	PSH	RST	SYN	FIN
0	1	0	0	0	0

Captura en Wireshark (vista simplificada)

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.100	93.184.216.34	TCP	60	51514 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
2	0.023456	93.184.216.34	192.168.1.100	TCP	60	80 → 51514 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
3	0.023789	192.168.1.100	93.184.216.34	TCP	60	51514 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0

Leyenda de flags:

SYN: Synchronize (sincronización)

ACK: Acknowledgment (acuse de recibo)

PSH: Push (empujar datos)

RST: Reset (reiniciar conexión)

FIN: Finish (finalizar conexión)

URG: Urgent (urgente)

Segmentación y reensamblado



Cuando una aplicación envía datos la capa de transporte lo divide en segmentos manejables.

El número de secuencia permite identificar cada segmento para que el receptor pueda reensamblarlos en el orden correcto.

De esta forma:

- Se puede lidiar con limitaciones físicas de la red (MTU)
- Evitar fragmentación IP
- Mejorar el control de errores
- Aumentar la eficiencia



UDP (User Datagram Protocol)



Características principales

- **No orientado a conexión**

Envía datagramas directamente. No hay handshake

- **No fiable**

La entrega no está garantizada y los segmentos pueden llegar dañados

- **No garantiza orden**

Los segmentos pueden llegar desordenados

- **Muy rápido**

Gracias a no tener que establecer un conexión y que es muy liviano.

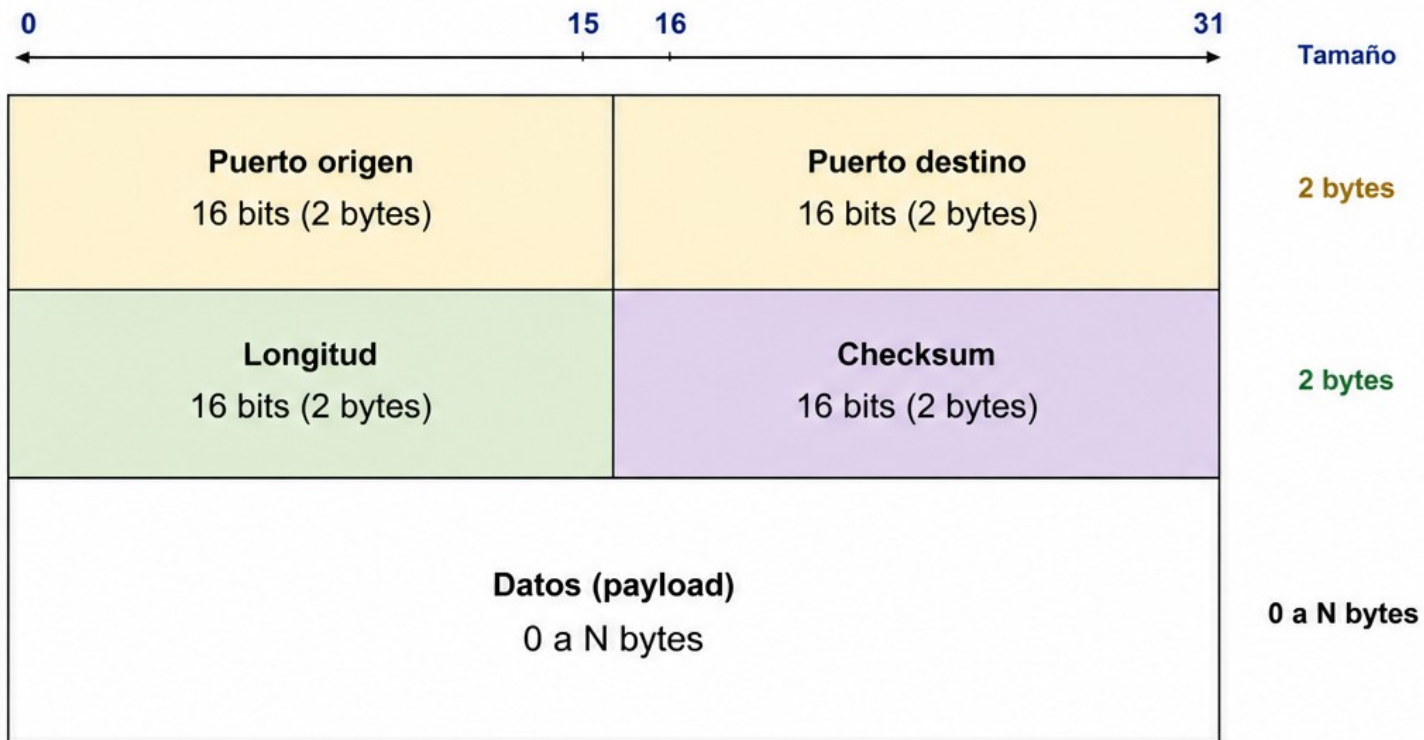
Ejemplos de uso: DNS, Streaming, VoIP, Juegos online



Formato de un segmento UDP



Formato de un segmento UDP



Descripción de los campos



Puerto origen:

Identifica el proceso o aplicación que envía el datagrama.



Puerto destino:

Identifica el proceso o aplicación que recibe el datagrama.



Longitud:

Indica el tamaño total del segmento UDP (cabecera + datos) en bytes. Valor mínimo: 8 (solo cabecera).



Checksum:

Suma de comprobación para detección de errores en la cabecera y los datos. Es obligatorio en IPv6 y opcional en IPv4 (si es 0, no se usa).



Datos (payload):

Información de la capa de aplicación. Puede tener 0 bytes de longitud.

Formato de un segmento UDP



- **Puerto fuente** (16 bits). Puerto de la máquina origen. Al igual que el puerto destino es necesario para identificar la conexión actual.
- **Puerto destino** (16 bits). Puerto de la máquina destino.
- **Longitud del mensaje** (16 bits). Especifica la longitud medida en bytes del mensaje UDP incluyendo la cabecera. La longitud mínima es de 8 bytes.
- **Suma de verificación** (16 bits, opcional en IPv4, obligatorio en IPv6). Suma de comprobación de errores del mensaje (Cabecera y Datos).
- **Datos**. Aquí viajan los datos que se envían las aplicaciones. Los mismos datos que envía la aplicación origen son recibidos por la aplicación destino después de atravesar toda la Red de redes.



NAT (Network Address Translation)



NAT permite que múltiples dispositivos en una red privada compartan una única IP pública, usando traducción de direcciones y puertos.

Se implementa habitualmente en routers.

Para su funcionamiento necesita modificar las direcciones IP en los paquetes

¿Qué problema resuelve NAT?



- Escasez de direcciones IPv4
- En una casa hay muchos dispositivos:
 - Móviles
 - Ordenadores
 - Smart Tvs

NAT permite que todos salgan con una sola IP pública.



¿Cómo funciona NAT?



Supón el siguiente caso:

PC interno: 192.168.1.20:52341

Servidor web: 93.184.216.34:443

Router IP pública: 80.25.10.7

El PC envía al router:

IP privada: 192.168.1.20:52341 → 93.184.216.34:443

Pero esa IP 192.168.1.20 es privada, no sirve en Internet. Entonces el router NAT cambia el origen:

80.25.10.7:60000 → 93.184.216.34:443

Ese 60000 podría ser cualquier puerto dinámico libre de la IP pública del router.

Guarda esta traducción en una tabla.



> -

[illegible]

93.184.216.34:443 → 80.25.10.7:60000

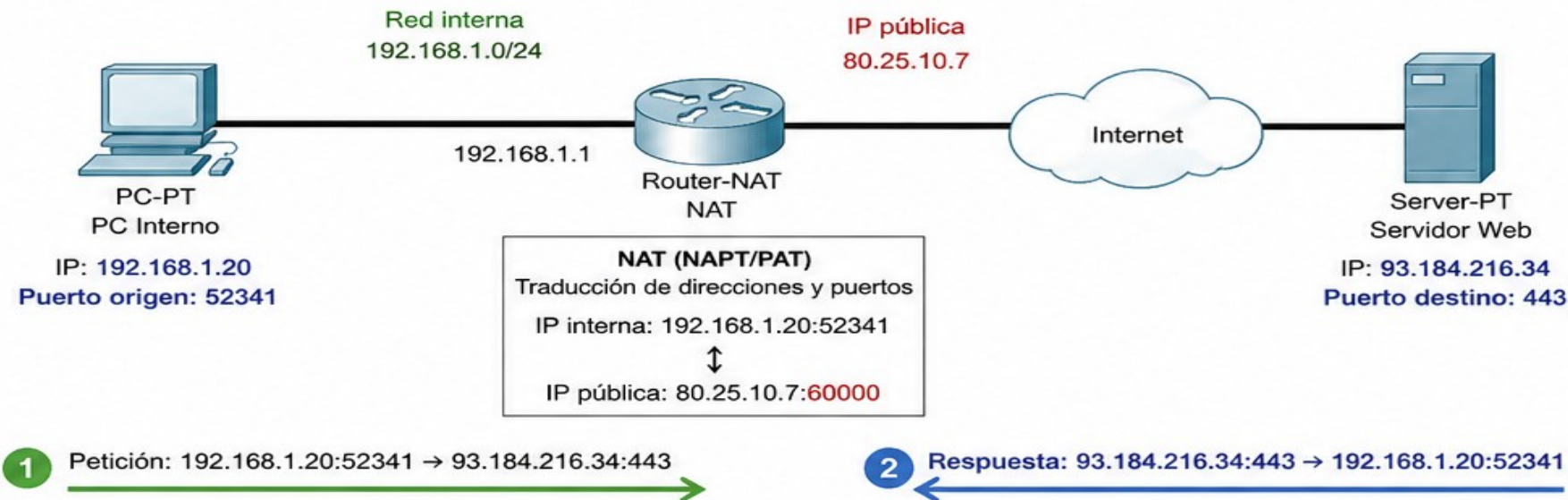
El router mira su tabla NAT y sabe:

80.25.10.7:60000 corresponde a 192.168.1.20:52341

Entonces reescribe el paquete y lo entrega dentro de la red:

93.184.216.34:443 → 192.168.1.20:52341

Ejemplo NAT



NAT Table (Router)

Inside Local IP	Inside Global IP
192.168.1.20	80.25.10.7:60000

PDU Information

Inbound PDU (from Server)

Src. IP: 192.168.1.20
Dst. IP: 93.184.216.34
Protocol: TCP

Outbound PDU (to Server)

Src. IP: 80.25.10.7
Dst. IP: 93.184.216.34
Protocol: TCP

Simulation

Reset Simulation

☒ Constant Delay

Captured to: (no captures)

Event List

El auge de QUIC



Uno de los cambios más importantes es la aparición de QUIC, desarrollado inicialmente por Google.

Los principales problemas de TCP son:

- Establecimiento de conexión lento (handshake)
- Head-of-line blocking (bloqueo por orden)
- Difícil de evolucionar (está en el kernel del SO)

QUIC viene a solucionar la mayoría de los problemas



¿Qué hace QUIC?



- Usa UDP como base
- Implementa encima:
 - Fiabilidad
 - Control de congestión
 - Seguridad (TLS integrado)

Es como un TCP moderno en espacio de usuario.



Ventajas clave



- **Conexión más rápida**
 - QUIC utiliza UDP y crea una conexión lógica (ligera) por encima.
- **Multiplexación sin bloqueos**
 - En TCP si un paquete se pierde bloquea todo
 - Con QUIC hay múltiples flujos independientes sin bloqueo entre ellos.
- **Seguridad integrada por defecto** (TLS integrado)
- **Adaptación a redes modernas** (móviles, WiFi, 5G)
 - Con QUIC puedes cambiar de WiFi a 4G/5G y la conexión no se rompe.

Por ello es la base de **HTTP/3** y cada vez más webs lo están adoptando.



TCP vs QUIC



Aspecto	TCP clásico	Tendencias actuales
Implementación	Kernel	User-space
Evolución	Lenta	Rápida
Seguridad	Opcional (TLS)	Integrada
Adaptación	Limitada	Alta
Multiplexación	Limitada	Avanzada

